📚 Documentación del Sistema de Gestión de Estudiantes en Python

1. Introducción

Este programa es un **sistema básico de gestión de estudiantes desarrollado en Python** que permite administrar información académica mediante una interfaz de consola.

El sistema utiliza **estructuras de datos como listas y diccionarios**, junto con **funciones y estructuras de control**, para realizar operaciones como:

- Registrar estudiantes
- Consultar la lista de estudiantes
- Buscar estudiantes
- Actualizar información
- Eliminar estudiantes

El objetivo del programa es **simular un pequeño sistema administrativo académico** para practicar conceptos fundamentales de programación en Python.

---

2. Estructura de datos utilizada

El sistema utiliza una **lista de diccionarios** para almacenar los estudiantes.

```
registros = [
    {"id": 1143245632, "nombre": "Randy", "edad": 19, "programa":
"sistemas", "estado": "activo"}
]
```

Explicación

- **Lista (`[]`)**
  Permite almacenar múltiples estudiantes.
- **Diccionario (`{}`)**
  Representa la información de un estudiante.

Campos de cada estudiante

Campo	Descripción
id	Número de identificación del estudiante

Campo	Descripción
nombre	Nombre del estudiante
edad	Edad del estudiante
programa	Programa académico
estado	Estado del estudiante (activo o inactivo)

Ejemplo de un estudiante:

```
{
"id": 1143245632,
"nombre": "Randy",
"edad": 19,
"programa": "sistemas",
"estado": "activo"
}
```

---

3. Funciones del sistema

El programa utiliza **funciones para organizar el código y reutilizar operaciones**.

3.1 Función registrar_estudiantes()

Objetivo

Permite **registrar un nuevo estudiante en el sistema**.

Código

```
def registrar_estudiantes(estudiantes):

    estudiantes = {
        "id": id,
        "nombre": nombre,
        "edad": edad,
        "programa": programa,
        "estado": estado
    }

    registros.append(estudiantes)

    print("Successful Student Registration")
```

Funcionamiento

1. Se crea un **diccionario con la información del estudiante**.
2. Se agrega el estudiante a la lista **registros** usando `append()`.
3. Se muestra un mensaje confirmando el registro.

Método utilizado

```
append()
```

Este método **agrega un elemento al final de una lista**.

Ejemplo:

```
lista.append(nuevo_elemento)
```

3.2 Función consultar_lista_estudiantes()

Objetivo

Mostrar todos los estudiantes registrados en el sistema.

Código

```
def consultar_lista_estudiantes(registros):

    if len(registros) == 0:
        print("The list is empty")
        return

    for e in registros:
        print(f"Id: {e['id']} | Nombre: {e['nombre']} | Edad: {e['edad']}
| Programa: {e['programa']} | Estado: {e['estado']}")
```

Funcionamiento

1. Se verifica si la lista está vacía con:

```
len(registros)
```

2. Si la lista está vacía, se muestra un mensaje.
3. Si contiene estudiantes, se usa un **bucle for** para recorrerlos.

Bucle utilizado

```
for e in registros
```

Este bucle recorre **cada estudiante de la lista**.

3.3 Función buscar_estudiantes()

Objetivo

Buscar un estudiante por su nombre.

Código

```
def buscar_estudiantes(registros, nombre):

    for e in registros:
        if e["nombre"].lower() == nombre.lower():
            return e

    return None
```

Funcionamiento

1. Se recorre la lista de estudiantes.
2. Se compara el nombre ingresado con el nombre guardado.
3. Se usa `.lower()` para evitar errores por mayúsculas o minúsculas.
4. Si se encuentra el estudiante, se retorna el diccionario.
5. Si no se encuentra, se retorna **None**.

Ejemplo:

```
resultado = buscar_estudiantes(registros, "Randy")
```

3.4 Función actualizar_informacion()

Objetivo

Actualizar la edad o el estado de un estudiante.

Código

```
def actualizar_informacion(registros, nombre, nueva_edad, nuevo_estado):

    estudiantes = buscar_estudiantes(registros, nombre)

    if estudiantes is None:
        print("Student not found")
        return
```

```
    if nueva_edad is not None:
        estudiantes["edad"] = nueva_edad

    if nuevo_estado is not None:
        estudiantes["estado"] = nuevo_estado

    print("Student updated successfully")
```

Funcionamiento

1. Se busca el estudiante usando la función **buscar_estudiantes()**.
2. Si el estudiante no existe, se muestra un mensaje.
3. Si existe:
    - Se actualiza la edad.
    - Se actualiza el estado.

Ejemplo de actualización:

```
estudiantes["edad"] = nueva_edad
```

3.5 Función eliminar_estudiante()

Objetivo

Eliminar un estudiante del sistema.

Código

```
def eliminar_estudiante(registros, nombre):

    estudiantes = buscar_estudiantes(registros, nombre)

    if estudiantes:
        registros.remove(estudiantes)
        print("Student delete")
    else:
        print("Student not found")
```

Funcionamiento

1. Se busca el estudiante.
2. Si existe:
    - Se elimina de la lista con `remove()`.
3. Si no existe:
    - Se muestra un mensaje.

Método utilizado

```
remove()
```

Este método **elimina un elemento específico de la lista**.

---

4. Menú principal del programa

El sistema utiliza un **bucle infinito** **`while True`** para mostrar el menú continuamente.

```
while True:
```

Menú

```
print("\n--- STUDENT MANAGEMENT ---")
print("1. Register new students")
print("2. Check the student list")
print("3. Find a student")
print("4. Update student information")
print("5. Delete students")
```

Cada opción permite ejecutar una función diferente.

---

5. Control de opciones del usuario

El programa utiliza **condiciones if/elif** para ejecutar la opción seleccionada.

Ejemplo:

```
if opcion == "1":
```

Dependiendo de la opción elegida:

Opción	Acción
1	Registrar estudiante
2	Ver lista
3	Buscar estudiante
4	Actualizar información
5	Eliminar estudiante

---

6. Manejo de errores

El programa usa **try y except** para evitar errores al ingresar datos.

Ejemplo:

```
try:
    edad = int(edad)
except ValueError:
    print("Invalid data")
```

Esto evita que el programa se detenga si el usuario escribe datos incorrectos.

---

7. Tecnologías y conceptos utilizados

Este sistema utiliza conceptos fundamentales de Python:

- Listas
- Diccionarios
- Funciones
- Condicionales (if)
- Bucles (for / while)
- Manejo de errores
- Métodos de listas

---

8. Posibles mejoras del sistema

El sistema puede mejorarse agregando:

- Validación de ID duplicado
- Búsqueda por ID
- Guardado de datos en archivos CSV
- Interfaz gráfica con Tkinter
- Base de datos (SQLite)

---

9. Conclusión

Este programa demuestra el uso de **estructuras básicas de programación en Python** para crear un sistema simple de gestión de estudiantes.

Permite aplicar conceptos importantes como:

- Organización del código con funciones
- Manipulación de listas y diccionarios
- Interacción con el usuario mediante consola
- Control de flujo del programa

Este tipo de ejercicios es fundamental para **desarrollar habilidades en programación y lógica computacional**.